

DataFrame of Mind

Data pipelines with Polars and PySpark · Data Minded Academy



Contents

- 01 Data and schemas

- 02 The engines: pandas, DuckDB, Polars

- 03 Expressing a query

- 04 How the engine executes it

- 05 Choosing an engine

- 06 PySpark



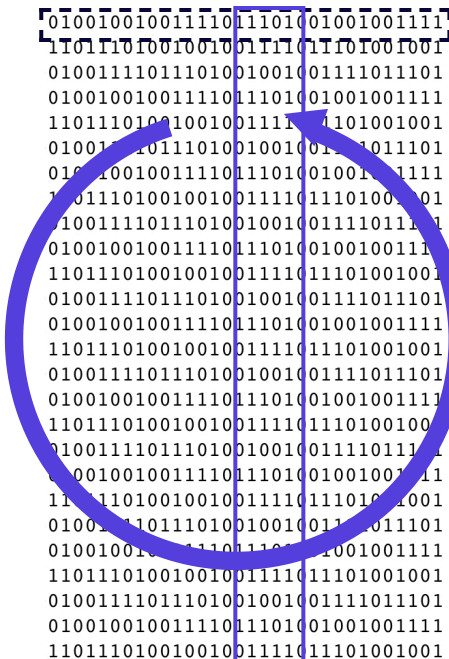
Data and schemas



Operational vs. Analytical Data

Operational ⚡

- Detailed events
- Fast and complete (ACID)
- Reads and writes **one record**, all of its fields



Analytical 🔍

- Descriptive and predictive
- Slow and selective
- Reads **one field** across every record

Operations capture reality event by event, analytics finds the patterns, and the patterns change how the next event is served.



Rows group records; columns group fields

Row-oriented

one line per record

id,name,weight_kg,age,heart_rate,country

8,Ana,210,4,76,CA

1042,Nanook,480,15,54,N0

73,Pipaluk,320,9,68,GL

615,Siku,405,12,61,CA

204,Nuka,260,6,72,US

981,Tala,505,18,52,RU

Give me record **615**

Column-oriented

one line per field

field,value1,value2,value3,value4,value5,value6

id,8,1042,73,615,204,981

name,Ana,Nanook,Pipaluk,Siku,Nuka,Tala

weight_kg,210,480,320,405,260,505

age,4,15,9,12,6,18

heart_rate,76,54,68,61,72,52

country,CA,N0,GL,CA,US,RU

Give me the highest **age**

SAME DATA

Same values, different order on disk, different work to retrieve them.



Exercise time: operational vs. analytical data

1-operational-vs-analytic/



A schema gives values meaning

A RECORD WITHOUT CONTEXT

1042, Nanook, 480, 15, 54, NO



add names and types

id	name	weight_kg	age	heart_rate	country
1042	Nanook	480	15	54	NO
Int64	String	Int64	Int64	Int64	String

Layout tells us where values are. **Schema** tells us what they are.



File formats combine **layout** and **schema**

Record-oriented

- **CSV / TSV**: text; schema supplied or inferred
- **Avro**: binary; writer schema stored with the data



Column-oriented

- **Parquet / ORC**: binary; schema and statistics stored in the file
- **Arrow IPC / Feather**: columnar interchange between tools



The format changes the physical representation, not what the data means.



Different formats, one **DataFrame** abstraction

id Int64	name String	weight_kg Int64	age Int64	heart_rate Int64	country String
8	Ana	210	4	76	CA
1042	Nanook	480	15	54	NO
73	Pipaluk	320	9	68	GL
615	Siku	405	12	61	CA

Row · one complete record

Column · one typed field across records

Libraries let us work with rows and typed columns, independent of the source format.



Bring in the Animals



An engine turns a query into **actual work**



A **query engine** reads bytes in whatever layout they arrive, applies the operations you asked for, and writes the result back out.



Three engines, one DataFrame

	 pandas	 DuckDB	 Polars
Born 🐣	2008 🇺🇸	2019 🇮🇪	2020 🇮🇪
What it is 📖	The original Python DataFrame library, and still the most widely taught.	A small analytics database that runs inside your Python process.	A newer DataFrame library, built for speed from the start.
You write 📝	Python	SQL	Python
Loved by ❤️	Data scientists 🧑🔬	Data analysts 🧑💻	Data engineers 🧑🔧

All three hand you the same DataFrame. What differs is how you **ask** for it.



The same question in **three dialects**

Average heart rate per bear, adults only.

 **pandas**

```
df = pd.read_parquet(path)
adults = df[df["age"] >= 4]
(adults
 .groupby("name")["heart_rate"]
 .mean())
```

 **DuckDB**

```
SELECT name, avg(heart_rate)
FROM 'measurements.parquet'
WHERE age >= 4
GROUP BY name
```

 **Polars**

```
(pl.scan_parquet(path)
 .filter(pl.col("age") >= 4)
 .group_by("name")
 .agg(
   pl.col("heart_rate").mean()
 )
 .collect())
```



Polars is a side project that got out of hand 🙌

- Main author: Ritchie Vink 🇳🇱, a structural engineer by education
- Started during the COVID-19 pandemic 😓, as an exercise in learning Rust
- No database career behind it, so no inherited assumptions: no index, expressions everywhere, Arrow from day one
- Full-time on Polars since July 2023, when he founded Polars Inc. to fund the work
- The company sells enterprise support and Polars Cloud



How to read a CSV with **polars**

```
pl.read_csv(  
    source: str | Path | IO[str] | IO[bytes] | bytes,  
    *,  
    has_header: bool = True,  
    columns: Sequence[int] | Sequence[str] | None = None,  
    new_columns: Sequence[str] | None = None,  
    separator: str = ',',  
    comment_prefix: str | None = None,  
    skip_rows: int = 0,  
    skip_lines: int = 0,  
    schema: SchemaDict | None = None,  
    schema_overrides: Mapping[str, PolarsDataType] | Sequence[PolarsDataType] | None = None,  
    null_values: str | Sequence[str] | dict[str, str] | None = None,  
    empty_string_is_null: bool = True,  
    infer_schema: bool = True,  
    infer_schema_length: int | None = 100,  
    ..., # there are many more arguments you can pass  
) -> DataFrame
```

A CSV may name its columns, never their **types**. Since Polars works with DataFrames, you will need to help it understanding the schema.



Demo time: a dirty, dirty CSV

`demo-reading-data/` and `2-csv-from-hell/`



Expressing a query



Imperative, declarative and functional styles

Imperative

```
seniors = []  
for row in rows:  
    if row["age"] >= 15:  
        seniors.append(row)
```

Step by step, like a recipe.

Declarative

```
SELECT name, weight  
FROM batch_measurements  
WHERE age >= 15
```

Describe what you want, let an optimised engine work out how.

Functional

```
vet = pl.read_parquet(path)  
seniors = vet.filter(  
    pl.col("age") >= 15  
)
```

Chain functions into a pipeline, no duplication, no mutation.



Projection, selection, rename, set operations and the whole family of joins.

Project, filter, rename, union

SQL

```
SELECT name, timestamp,  
       temperature AS temp_c  
FROM (  
  SELECT name, timestamp,  
         temperature  
  FROM sensor  
  UNION ALL  
  SELECT name, timestamp,  
         NULL AS temperature  
  FROM vet  
)  
WHERE temperature > 40
```

`UNION` deduplicates, `pl.concat` does not. `UNION ALL` is the honest translation.

Polars

```
readings = (  
  pl.concat(  
    [                                     # a list, not *args  
      sensor.select(  
        "name", "timestamp", "temperature",  
      ),  
      vet.select("name", "timestamp"),  
    ],  
    how="diagonal",                      # gaps become null  
  )  
  .rename({"temperature": "temp_c"})  
  .filter(pl.col("temp_c") > 40)  
)
```



Beyond relational algebra: contexts and expressions

Polars adds its own DSL on top of the relational engine. An **expression** is a tree of operations describing how to build one or more Series. Expressions are always evaluated inside a **context**: `select`, `with_columns`, `filter` and `group_by`.

```
vet = pl.read_parquet("data/batch_measurements.parquet")

query = vet.with_columns(                # context
    (pl.col("weight") / pl.col("age")).alias("kg_per_year") # expression
).filter(                               # context
    pl.col("life_stage") == "SENIOR"    # expression
).select(                               # context
    pl.col("name"), pl.col("kg_per_year").round(1) # expression
)
```



One expression, four contexts

The expression never changes: `heaviest = pl.col("weight").max()`. The context decides what comes back.

CONTEXT	ROWS OUT	WHAT YOU ASKED FOR
<code>vet.select(heaviest)</code>	1	One answer for the whole table.
<code>vet.with_columns(heaviest)</code>	3 291	The same answer stapled onto every row, ready to compare against.
<code>vet.group_by("life_stage").agg(heaviest)</code>	4	One answer per group.
<code>vet.filter(pl.col("weight") == heaviest)</code>	1	The row that holds the answer, not the value.

You are not calling functions on data. You are handing the engine a **description** and a place to evaluate it.



Polars comes with a big bag of **batteries** included

Column selectors

```
import polars.selectors as cs

vet.select(cs.numeric())
vet.select(cs.starts_with("vet"))
```

Meta-queries over the schema,
instead of hard-coded column lists.

Type namespaces

```
vet.with_columns(
    pl.col("name")
        .str.to_titlecase()
)
vet.with_columns(
    pl.col("timestamp")
        .dt.year()
)
```

Type-specific functions live in
`.str`, `.dt`, `.list` and `.struct`.

Testing helpers

```
from polars.testing import (
    assert_frame_equal)

assert_frame_equal(df1, df2)
# AssertionError: columns
# ['foo', 'bar', 'baz'] in left
# DataFrame, but not in right
```

Frame and series comparisons that
fail with a readable message.



Exercise time: relational algebra

3-basic-transforms/



To SQL or not to SQL?

SQL wins on

- Readability
- Getting stuff done
- Easy to learn, everyone speaks it



A DataFrame API wins on

- Abstraction (functions, modules)
- Control flow (if, for, while)
- Testing
- Ecosystem (linting, typing, packaging)



Windowing and aggregations

Window: one row in, one row out

```
vet.with_columns(  
  pl.col("weight")  
    .mean()  
    .over("life_stage")  
    .alias("avg_for_stage")  
)  
# 3 291 rows in, 3 291 rows out
```

Calculate a value over a group and add it to every record of that group.

Aggregation: one group, one row

```
vet.group_by("life_stage").agg(  
  pl.col("weight")  
    .mean()  
    .alias("avg_for_stage")  
)  
# 3 291 rows in, 4 rows out
```

Calculate a value over a group and return one record per group.

Same expression, same grouping. The **context** decides whether you keep your rows.



Aggregations that return a row, not a value

```
# the whole latest reading per bear, not just the latest timestamp
vet.group_by("name").agg(pl.all().sort_by("timestamp").last())

# which bear was most active, per life stage per year
vet.group_by("life_stage", pl.col("timestamp").dt.year().alias("year")).agg(
    pl.col("name").sort_by("daily_steps").last().alias("most_active"),
    pl.col("daily_steps").max(),
)

# an ordered window: no .sort() beforehand, the window sorts itself
vet.with_columns(
    pl.col("weight").rolling_mean(window_size=3).over("name", order_by="timestamp")
)
```

`sort_by` inside an aggregation is the answer to "which one", and `order_by` is what makes a window trustworthy.

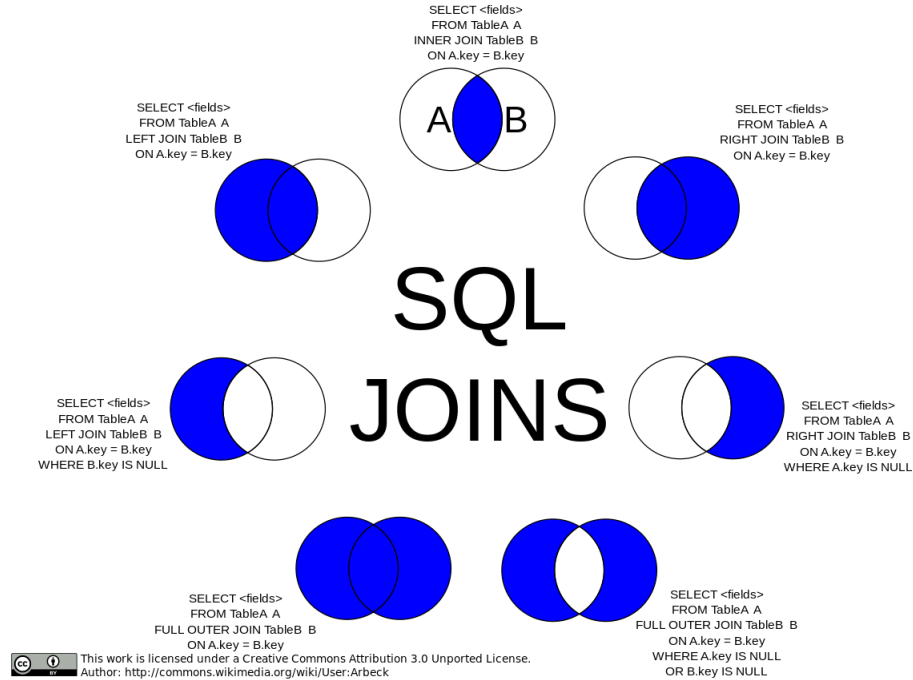


Exercise time: windowing and aggregations

4-window-aggregations/



The standard join types



Polars supports all **standard** join operations

```
DataFrame.join(
    other: DataFrame,
    on: str | Expr | Sequence[str | Expr] | None = None,
    how: JoinStrategy = 'inner',          # left, right, full, semi, anti, cross
    *,
    left_on: str | Expr | Sequence[str | Expr] | None = None,
    right_on: str | Expr | Sequence[str | Expr] | None = None,
    suffix: str = '_right',              # what happens to colliding column names
    validate: JoinValidation = 'm:m',    # 1:1, 1:m, m:1
    join_nulls: bool = False,
    coalesce: bool | None = None,
) -> DataFrame
```

```
sensor.join(vet, on="name")                # 158 016 x 3 291 -> 90 475 536 rows
sensor.join(vet, on="name", validate="m:1")
# ComputeError: join keys did not fulfill m:1 validation
```

validate turns a silent row explosion into an error. It costs one argument and it is the cheapest test in this course.



... and even non-standard joins

```
sensor.join_asof(  
  vet.select("name", "timestamp", "vet_health_check"),  
  on="timestamp",           # both sides must be sorted on this  
  by="name",                # exact match on the bear, nearest match on the time  
  strategy="backward",      # the most recent verdict at or before the reading  
  tolerance="7d",           # beyond this, null instead of a stale answer  
)  
# 158 016 rows in, 158 016 rows out
```

Match on the nearest key rather than an exact one. Useful for time series: sensor readings joined to the most recent configuration change.

`join_where` is the general case: join on any predicate, not just equality.



Exercise time: joins

5-joins/

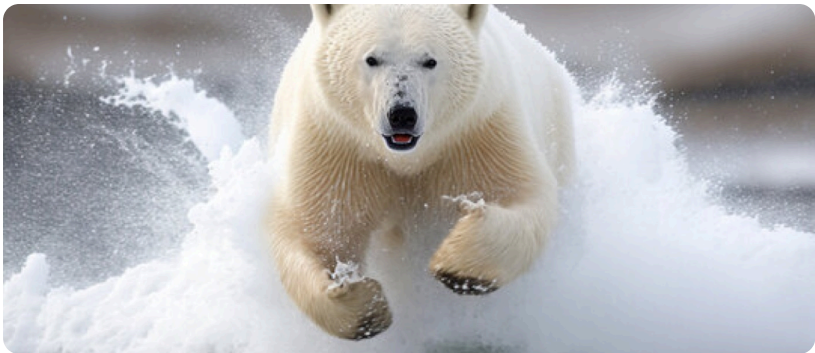


How the engine executes it



Lazy vs. **eager** evaluation

Eager: `read_csv`



Every step runs the moment you write it. Fine for exploration, wasteful for pipelines.

Lazy: `scan_csv`



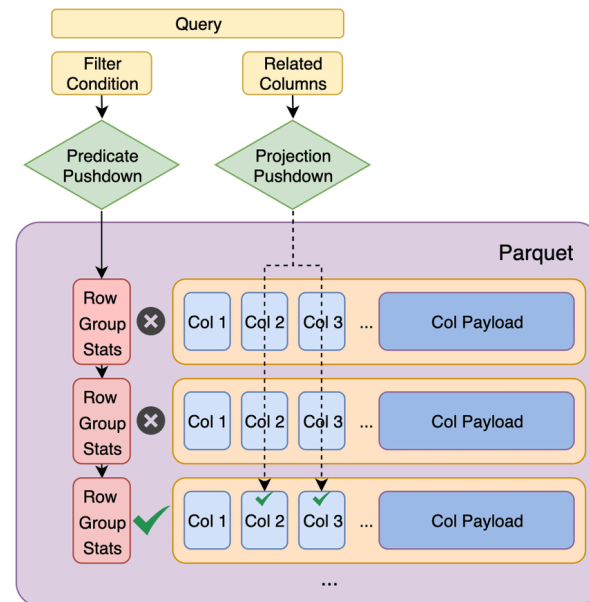
Nothing runs until `.collect()`. The optimiser sees the whole query and rewrites it.



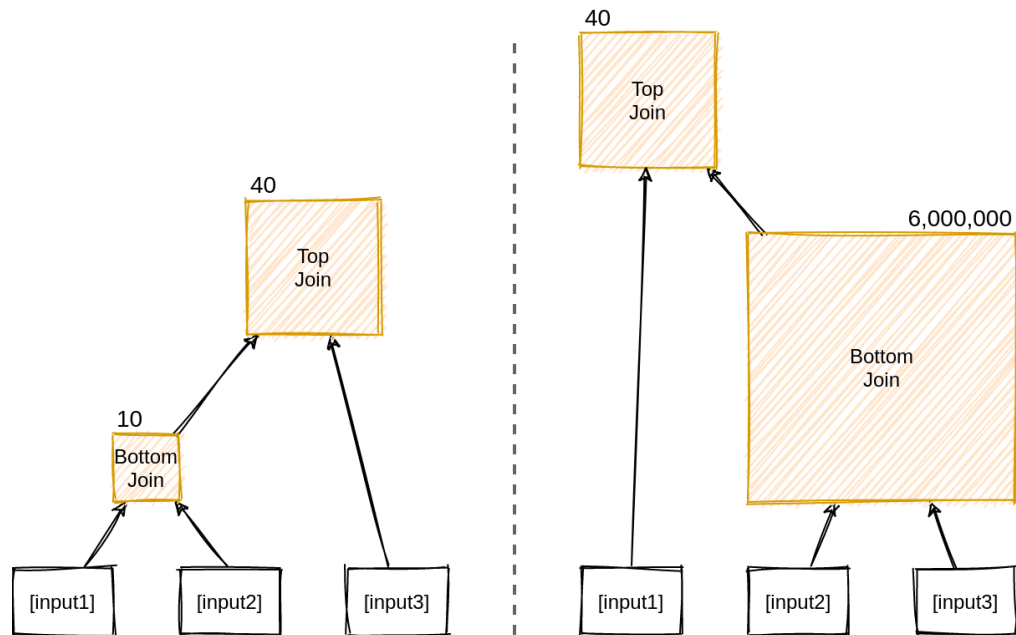
Some optimisations: pushdown

- **Predicate pushdown:** filter rows as early as possible
- **Projection pushdown:** drop columns as early as possible
- **Slice pushdown:** read only the rows a **head** or **slice** actually needs

Applied while reading the data. Parquet stores min/max statistics per row group, so the engine skips whole groups and those rows never enter memory at all.



Some optimisations: join reordering



Same three inputs, same result, 6 million intermediate rows of difference.



Polars streaming: when you are resource-constrained

Instead of processing the data all at once, Polars can execute the query in batches, which lets you process datasets larger than memory.

```
q1 = (  
    pl.scan_csv("car.csv")  
    .filter(pl.col("price") < 30_000)  
    .group_by("brand")  
    .agg(pl.col("max_speed").max())  
)  
df = q1.collect(engine="streaming")
```

What streams

Most of the API streams: `scan_csv`, `scan_parquet`, `scan_ipc`, `select`, `with_columns`, `filter`, `slice`, `group_by`, `join`, `unique`, `sort`, `explode`, `unpivot`.

Anything that cannot stream falls back to the in-memory engine on its own, so a query never fails for this reason. To see which part does what:

```
show_graph(plan_stage="physical",  
engine="streaming").
```



A bit unread is a bit less to process



Pushdown is the engine skipping bytes for you. It can only skip what the layout allows, so partition along the column you filter on first.



Demo: hive-partitioned vs. plain Parquet



When SQL doesn't cut it: user defined functions

map_rows

```
df.map_rows(  
    lambda t: (t[0] * 2, t[1] * 3)  
)
```

Every row as a tuple. Column names are lost, and it is the slowest of the three.

map_batches

```
pl.col("features").map_batches(  
    lambda s: model.forward(  
        s.to_numpy()  
    )
```

The whole Series at once, so NumPy and PyTorch run once per column.

map_elements

```
pl.col("a").map_elements(  
    lambda x: x * 2,  
    return_dtype=pl.Int64,  
)
```

One value at a time. Pass `return_dtype` or Polars infers it from the first result.

Every UDF drops out of the optimised engine and back into the Python interpreter, so check for a builtin first.



Exercise time: UDFs

6-udf/



Big Data?



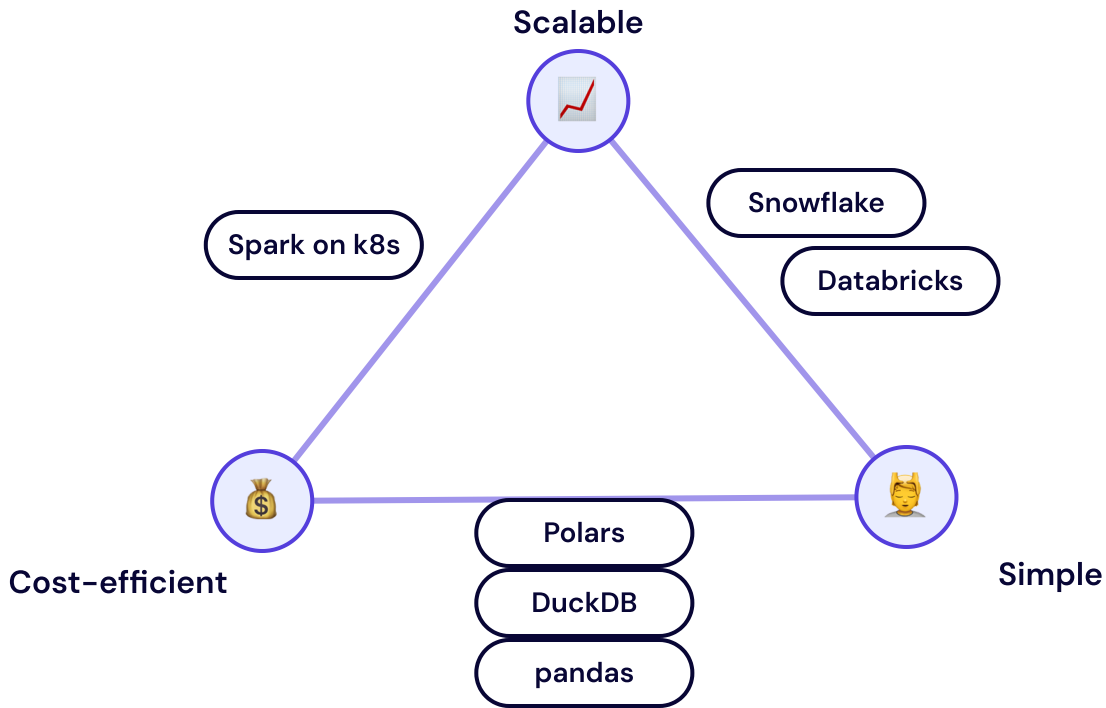
The data processing landscape over the years



- 📈 First wave optimised for **scale**: commodity machines, open source, structured and semi-structured data
- 🐼 Second wave optimised for **simplicity**: managed platforms hid the cluster
- 💰 Third wave optimised for **cost**: Kubernetes brought the cluster back, cheaper

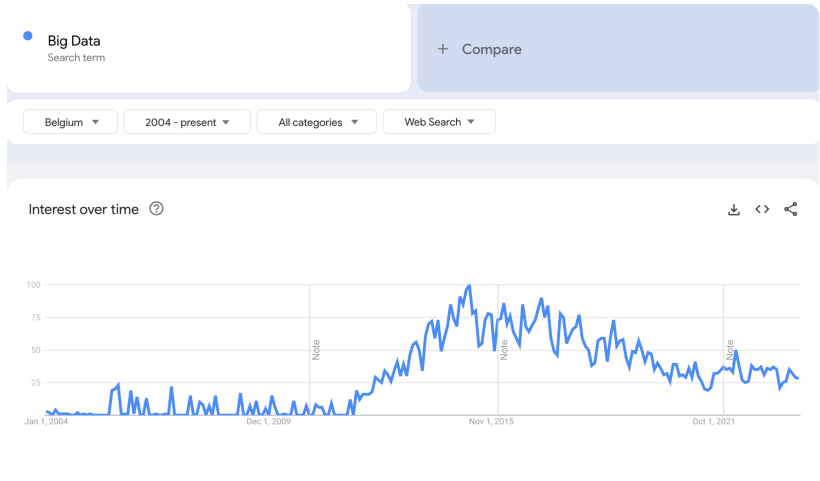


Emerging single node processing technologies



Interest in big data, and what one machine **now** holds

Interest in 'big data' is past its peak



One machine now holds 12 TB of RAM

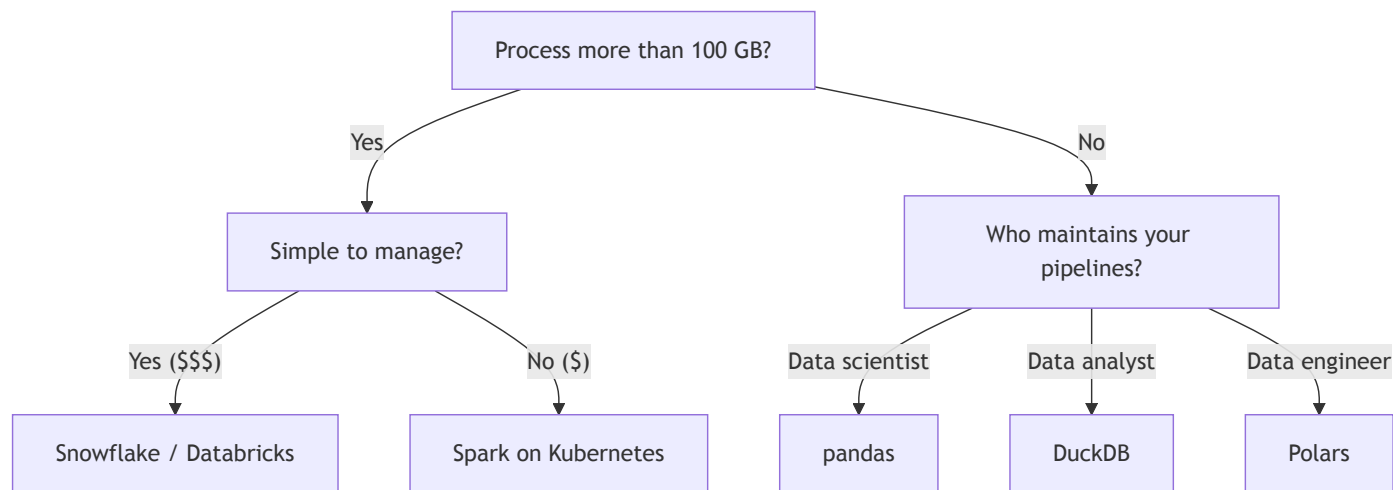
Instance name ▼	On-Demand hourly rate ▼	vCPU ▼	Memory ▼
u-12tb1.112xlarge	\$109.20	448	12288 GiB
u-6tb1.112xlarge	\$54.60	448	6144 GiB
u-6tb1.56xlarge	\$46.40391	224	6144 GiB
x2iedn.32xlarge	\$26.676	128	4096 GiB
x2iedn.metal	\$26.676	128	4096 GiB
x1e.32xlarge	\$26.688	128	3904 GiB
u-3tb1.56xlarge	\$27.30	224	3072 GiB
x2iedn.24xlarge	\$20.007	96	3072 GiB



You can't please all of the people all the time,
luckily there is an **escape hatch**



When do I use **what**?



Apache Spark



The same query in PySpark and Polars

PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.window import Window
import pyspark.sql.functions as sf

spark = SparkSession.builder.getOrCreate()

df = (
    spark.read.csv("measurements.csv")
    .withColumn("year", sf.year("timestamp"))
    .withColumn("max_weight_per_year",
                sf.max("weight")
                  .over(Window.partitionBy("year")))
    )

df.collect()
```

Polars

```
import polars as pl

df = (
    pl.scan_csv("measurements.csv")
    .with_columns(
        pl.col("timestamp").dt.year()
        .alias("year")
    )
    .with_columns(
        pl.col("weight").max().over("year")
        .alias("max_weight_per_year")
    )
)

df.collect()
```

Both express the same operations, and both are lazy. The rest of this section covers what changes when the work runs on a cluster instead of in one process.



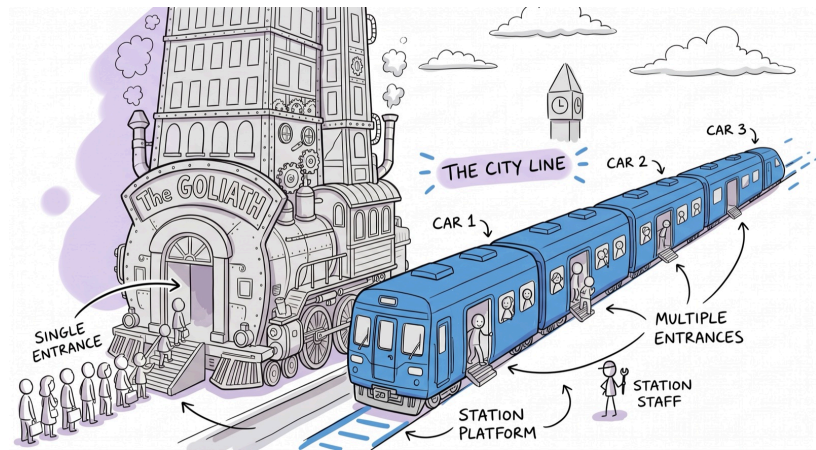
Large jobs are limited by I/O, and a cluster adds bandwidth

One machine

- One network card, one set of disks: a few GB/s at best
- 10 TB at 1 GB/s: three hours before any computation starts
- The CPUs idle, waiting for bytes

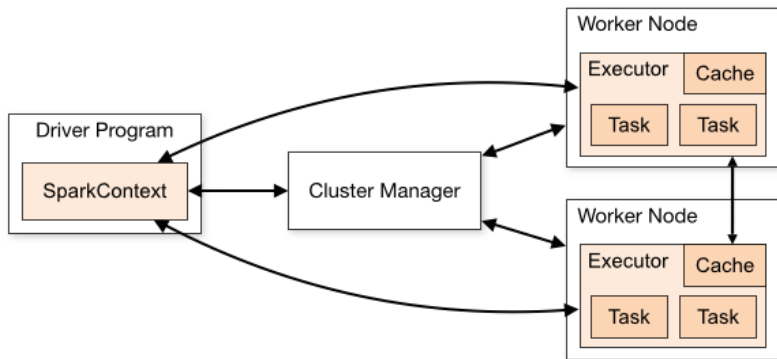
A cluster

- Every executor reads its own slice, so bandwidth grows with the machine count
- 10 TB across 60 executors: minutes, if the object store keeps up
- Memory scales the same way



A submitted job runs on a driver and its executors

Scaling out means adding machines rather than buying a bigger one. This is the machinery that makes that possible.



Your program runs in the driver. The cluster manager allocates executors on worker nodes, and the driver ships the code and the tasks to them.

```
$ ./bin/spark-submit \
  --master k8s://https://<host>:<port> \
  --deploy-mode cluster \
  --name spark-pi \
  --conf spark.executor.instances=5 \
  --conf spark.kubernetes.container.image=<image> \
  local:///path/to/examples.jar
```

`spark-submit` sets `SPARK_HOME` and `JAVA_HOME` and hands your program to the launcher.

Note the arrow between the workers: executors talk to each other, not only to the driver. The next slide is about that arrow.



Wide transformations **shuffle** data across the network

Narrow: no data moves

`select`, `filter`, `withColumn`, `union`

Each executor works on the partitions it already holds.
Spark chains these into one stage.

Wide: rows meet on one executor

`groupBy`, `join`, `distinct`, `sort`, `repartition`

All rows with the same key have to end up in the same place, so Spark writes shuffle files and moves them over the network. That ends a stage and starts the next.

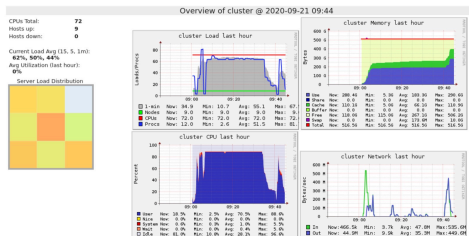
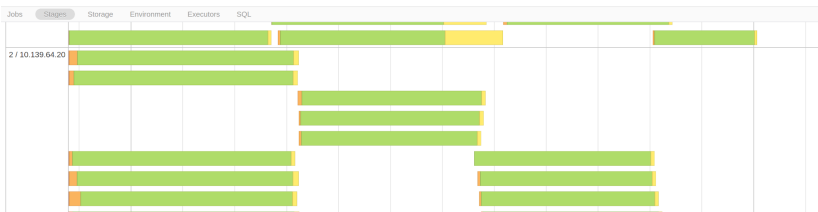
```
df.filter(sf.col("age") > 18).groupBy("country").count().explain()
# AdaptiveSparkPlan isFinalPlan=false                (abridged)
# +- HashAggregate(keys=[country], functions=[count(1)])
#   +- Exchange hashpartitioning(country, 200)         <- the shuffle
#       +- HashAggregate(keys=[country], [partial_count(1)])
#           +- Filter (age > 18)
```

No Polars equivalent: in one process a `group_by` is a hash table, not a network transfer.

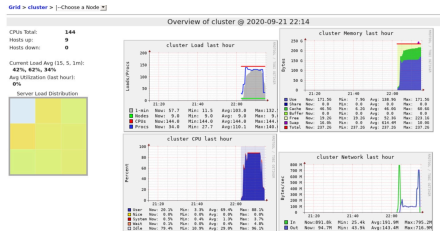


Partition count decides how much of the cluster stays busy

72 vCPUs, idle at the tail



Partitions matched to cores



The 200 in that `Exchange` is the default of `spark.sql.shuffle.partitions`, applied to every shuffle whatever your data or cluster size. Make it a multiple of the cluster cores. Since Spark 3.2 adaptive execution coalesces them at runtime, so treat 200 as a starting point.



Transformations are **lazy**, actions trigger execution

```
import pyspark.sql.functions as sf

df1 = spark.range(3)
df2 = df1.withColumn("foo", sf.lit("bar"))
df3 = df2.withColumn("bar", sf.col("id") < 2)
df4 = df3.filter(sf.col("id") != 1)
df5 = df4.select("foo", "bar", "id",
                 (sf.col("id") + 2).alias("plus2"))
df6 = df5.drop("id")
df7 = df6.withColumnRenamed("bar", "lessthan2")

df7.printSchema()    # free: bookkeeping only
print(df7.count())   # action: the plan runs
df7.show()           # action
```

Polars taught you this with `scan_csv` and `collect`. Spark works the same way: transformations build a plan, actions run it.

Two things are different once the plan runs on a cluster:

- ⚠ Errors surface at the **action**, not at the line that caused them, and they arrive as a Java stack trace from an executor.
- `DataFrame.explain()` shows the physical plan, including every `Exchange`.



Chaining drops the intermediate names, **transform** makes steps testable

Chain the calls

```
df = (
    spark.range(3)
    .withColumn("foo", sf.lit("bar"))
    .withColumn("bar", sf.col("id") < 2)
    .filter(sf.col("id") != 1)
    .select("foo", "bar", "id",
            (sf.col("id") + 2).alias("plus2"))
    .drop("id")
    .withColumnRenamed("bar", "lessthan2")
)
```

No `df1` to `df7` to keep straight, and no filtering `df3` when you meant `df4`.

Name the steps with transform

```
def add_flags(df: DataFrame) -> DataFrame:
    return (df
        .withColumn("foo", sf.lit("bar"))
        .withColumn("bar", sf.col("id") < 2))

def drop_middle(df: DataFrame) -> DataFrame:
    return df.filter(sf.col("id") != 1)

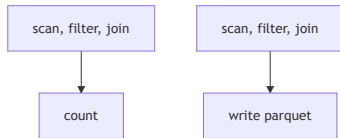
df = (spark.range(3)
    .transform(add_flags)
    .transform(drop_middle))
```

A `DataFrame -> DataFrame` function is testable on three rows, no cluster involved. **transform** is PySpark's pipe; Polars spells it `.pipe()`.

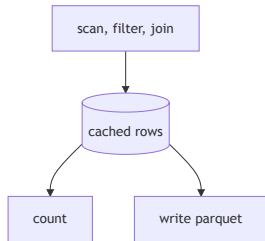


A DataFrame used twice is computed twice, unless you **cache** it

Two actions, two full recomputations



Cached: computed once, read twice



```
enriched = (spark.read.parquet("events")
    .filter(sf.col("year") == 2024)
    .join(dim_user, "user_id")
    .cache())           # lazy too: the first action fills it
enriched.count()       # computes, then keeps the rows
enriched.write.parquet("out") # reads the cache instead
```

Only worth it when one DataFrame feeds more than one action. Call `unpersist()` when the branch is done.



A UDF runs Python per row, **outside** Spark's optimiser

```
import pyspark.sql.functions as sf
from pyspark.sql.types import IntegerType

def square(x):
    return x**2

square_as_udf = sf.udf(
    square, returnType=IntegerType())
df.withColumn("squared", square_as_udf("id"))
```

You must declare the return type: the optimiser cannot infer it from Python.

In Polars a UDF drops from Rust into Python in the same process. In Spark, every row is serialised out of the JVM into a separate Python worker and back again.

That round trip is why `pandas_udf` exists: it hands the worker an Arrow batch instead of a row, so the crossing happens once per batch.

The rule from the Polars half still holds, only more so: look for a builtin first.



One core, four libraries: SQL, streaming, MLlib, GraphX



All four sit on the Spark core, whose abstraction is the resilient distributed dataset (RDD). The core also handles task dispatch, scheduling and I/O.

They share one API and one deployment, in Scala, Java, Python, R and SQL with almost identical names. Training a model on a batch and then scoring a stream needs no second system.



Batch and streaming share the same transformations

Batch: aggregate a database table

```
df = (spark
    .read
    .format("jdbc")
    .option("url", url)
    .option("dbtable", "people")
    .load()
)

counts_by_age = df.groupBy("age").count()
```

Streaming: count words as they arrive

```
lines = (spark
    .readStream
    .format("socket")
    .option("host", host)
    .option("port", port)
    .load()
)

words = lines.select(
    explode(split(lines.value, " ")).alias("word"))
word_counts = words.groupBy("word").count()

query = (word_counts.writeStream
    .format("console").start())
```

Swap `read` for `readStream` and the business logic is identical: `select`, `groupBy`, `count`.



SQL and the DataFrame API compile to the **same plan**

Both go through the same optimiser and run on the same executors. Neither is faster, and neither is "more distributed" than the other, so the choice is about maintaining the code.

Write it in SQL

```
with open("min_max_by_user.sql") as fh:
    query = fh.read()
spark.sql(query)
```

- Everyone on the team reads it
- Says what, not how
- No second API to learn

Write it in Python

- Functions and modules instead of copy and paste
- Branches and loops
- Unit tests on the transformation
- Type checking and linting before you deploy
- Machine learning in the same program

Spark distributes the work either way, so scale is not the tiebreaker.



Running it in practice



Polars ships as a **wheel**; PySpark needs a **JVM**

Polars

```
uv add polars
```

- One wheel with a Rust binary inside
- No runtime to install, no services to start
- The same code runs in a notebook, a Lambda and a CI job

PySpark

```
uv add pyspark          # ~300 MB  
apt install openjdk-17-jre # + JAVA_HOME
```

```
spark = (SparkSession.builder  
    .master("local[*]")          # one machine  
    .config("spark.sql.shuffle.partitions", 200)  
    .getOrCreate())
```

- `local[*]` for your laptop, a cluster manager for anything else
- Settings live in `conf/spark-defaults.conf`, this repo ships one

The cost of the second engine is the runtime around it: a JVM of the right version, a cluster manager, and configuration.



Connectors need matching JARs and configuration

AWS S3 outside EMR

```
config = {
    "spark.jars.packages":
        "org.apache.hadoop:hadoop-aws:<version>",
    "spark.hadoop.fs.s3a.aws.credentials.provider":
        "org.apache.hadoop.fs.s3a"
        ".SimpleAWSCredentialsProvider",
}
conf = SparkConf().setAll(config.items())
spark = SparkSession.builder.config(
    conf=conf).getOrCreate()

df = spark.read.csv(f"s3a://{BUCKET}/{KEY}",
                    header=True)
```

Snowflake

```
snowflake_pkgs = [
    "net.snowflake:snowflake-jdbc:<version>",
    "net.snowflake:spark-snowflake_2.12:<version>",
]
config = {
    "spark.jars.packages": ", ".join(snowflake_pkgs),
}
conf = SparkConf().setAll(config.items())
spark = SparkSession.builder.config(
    conf=conf).getOrCreate()
```

Versions have to line up: `hadoop-aws` against the Hadoop libraries in your Spark install, `spark-snowflake` against your Spark version. Then come the authentication options. Polars reads `s3://` with `pip install polars[aws]`.



Readers and writers handle formats, folders and partitioning

```
frame = spark.read.csv(
    str(csv_file_path), sep=";", header=True)

frame = (
    spark.read
    .options(header="true", sep=";", inferSchema=True)
    .format("csv")
    .load(str(csv_file_path))
)

frame.write.parquet(
    "some_location",
    mode="overwrite",
    partitionBy="country",
)
```

Hive-partitioned output

```
├─ date=2021-10-08
│   ├── train_id=200
│   │   ├── part-00000-6d...
│   │   └── part-00001-6d...
│   ├── train_id=231
│   │   ├── part-00000-6d...
│   │   └── part-00001-6d...
│   └── train_id=4361
│       ├── part-00000-6d...
│       └── part-00001-6d...
├─ date=2021-10-09
│   └── train_id=200
```

One file per partition per writer, in folders named after the column value. The same layout Polars read from earlier: partition on what you filter on first.



When to reach for Spark

Spark earns its keep

- The data does not fit one machine, and streaming it through does not help
- You already run a platform that hosts it: Databricks, EMR, Kubernetes
- You need batch, streaming and ML behind one API
- A job runs long enough that losing a node matters

A cluster costs a JVM, configuration, shuffles and idle CPUs, whether or not the data needs the scale. The decision tree from the first half still applies.

Stay on one node

- It fits in memory, or streams through a single machine
- You want a fast local loop and a short install
- The team is small and the pipeline is simple
- Cost per query matters more than peak scale



What we covered

1. **Same data, different storage.** Operational and analytical systems hold the same facts. Storing them row by row or column by column is what decides which questions are cheap to ask.
2. **Let the engine do the heavy lifting.**
 - It turns what you asked for into a plan, then reorders joins and pushes filters and column selections down into the scan, so it reads less.
 - You help it by storing data columnar and partitioning along the questions you actually ask. It can only skip what the layout allows.
3. **Relational algebra has lasted.** Project, filter, join, aggregate, window. SQL and the DataFrame API are two ways of writing the same operations.
4. **One node goes a long way.** For most pipelines a single machine is simpler, cheaper and quicker to iterate on. Reach for Spark when the data is genuinely large.





Thank you

